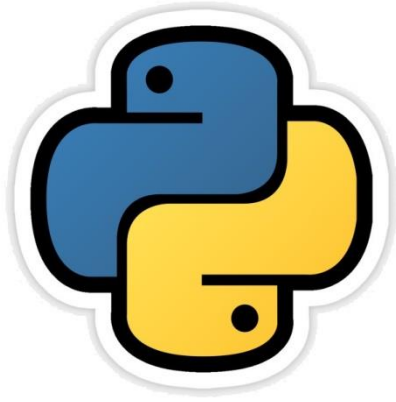




Recursion

सीबीएसई पाठ्यक्रम पर आधारित

कक्षा -12

अध्याय -6



द्वारा:

संजीव भदौरिया

स्नातकोत्तर शिक्षक (संगणक विज्ञान)

के० वि० बाराबंकी (लखनऊ संभाग)

Recursion - Introduction

- Recursion एक प्रकार की प्रोग्रामिंग तकनीकी है जिसके अंतर्गत कोई भी function, प्रत्यक्ष (directly) अथवा अप्रत्यक्ष (indirectly), स्वयं को ही कॉल करता है ।
- “Recursion refers to a programming technique in which a function calls itself either directly or indirectly.”
- कई प्रोग्राम्स में आपने देखा होगा की एक function किसी दूसरे function को निम्न प्रकार कॉल कर सकता है

```
def A() :  
    B()
```

यह एक सामान्य function के अन्दर function की calling है ।

Recursive Function

- अब यदि कोई function स्वयं को ही कॉल करे तो यह recursion है -

```
def A():  
    A()
```

यहाँ A() स्वयं को ही कॉल कर रहा है तो यह recursion है।

```
def Sum():  
    Sum()
```

यहाँ Sum() स्वयं को ही कॉल कर रहा है तो यह recursion है।

```
def fact(n):  
    if n==1:  
        return 1  
    else:  
        return (n*fact(n-1))
```

यहाँ fact() function खुद को ही कॉल कर रहा है।

```
print("The factorial is", fact(5))
```

== RESTART: C:\Users\
The factorial is 120

Recursive Function

- Recursion दो प्रकार का हो सकता है

- Direct recursion

```
def A() :  
    A()
```

यहाँ A () स्वयं को ही कॉल कर रहा है तो यह recursion है ।

- Indirect recursion

```
def A() :  
    B()  
  
def B() :  
    A()
```

यहाँ A () function में B () और B() function में A() को कॉल किया गया है

Recursive Function

- Recursion वह तकनीकी है जिसके द्वारा हम बड़ी बड़ी computational problems को हल करते हैं और इसके लिए बार बार उसी procedure को apply करके छोटी problem तक घटा कर ले आते हैं ।
- एक recursive procedure के दो हिस्से होते हैं -
 1. एक या अधिक base case
 2. एक recursive step
- एक recursive code को यह पता होना चाहिए की recursion को कहाँ रोकना है ।
- अन्यथा एक अनंत loop चलेगा और error आजायेगी ।

Recursive Function

- निम्न कोड पर ध्यान दें तो पता चलेगा की इसका आउटपुट अन्त विहीन होगा | यह एक infinite loop की तरह काम करेगा

```
def func1():  
    print("Hello func2")  
    func2()  
  
def func2():  
    print("Yes func1")  
    func1()  
  
func1()
```

```
Yes func1  
Yes func1  
Yes func1  
Yes func1  
Yes func1
```

यह आउटपुट लगातार आता रहेगा और python इसके लिए एक error generate करता है | अतः यह एक sensible recursive कोड नहीं है |

RecursionError: maximum recursion depth exceeded

Recursive Function

- सही और sensible recursive कोड वह होता है जो निम्न ज़रूरतों को पूरा करता है -
- इसमें एक case का होना आवश्यक है जिसके अंतर्गत recursive calling की जाती है | इस base case को दिए गए parameter / argument के लिए reachable होना चाहिए | इसे stoping case भी कह सकते हैं और यह अत्यंत आवश्यक होता है |
- इसमें एक recursive case होता है जिसमे function खुद को ही कॉल करता है |

Recursive Function

- निम्न प्रोग्राम को देखिये -

```
#program to print sum of n numbers
```

```
def sum_of_n(num):
```

```
    print("In this Pass num is : ", num)
```

```
    if num==1:
```

```
        return 1
```

```
    else:
```

```
        return (num+sum_of_n(num-1))
```

Base Case

RecursiveCase

```
#Main Program
```

```
n=int(input("Enter a number"))
```

```
sum=sum_of_n(n)
```

```
print("The sum of n numbers", sum)
```

```
= RESTART: C:/Users/SHAUR
```

```
Enter a number5
```

```
In this Pass num is : 5
```

```
In this Pass num is : 4
```

```
In this Pass num is : 3
```

```
In this Pass num is : 2
```

```
In this Pass num is : 1
```

```
The sum of n numbers 15
```


Recursive Function

- इस प्रोग्राम में execution ऐसे हुआ है -

sum_of_n(5)

= 5 + sum_of_n(4)

= 5 + (4 + sum_of_n(3))

= 5 + (4 + (3 + sum_of_n(2)))

= 5 + (4 + (3 + (2 + sum_of_n(1))))

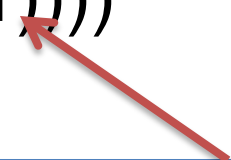
= 5 + (4 + (3 + (2 + 1)))

= 5 + (4 + (3 + 3))

= 5 + (4 + 6)

= 5 + 10

= 15



जैसे ही Base Case की condition true होती है वैसे ही 1 return होकर function समाप्त हो जाता है और पुनः स्वयं को कॉल नहीं कर पाटा है।

Recursive Function कैसे कार्य करता है ?

- इस प्रकार के execution में कंप्यूटर एक stack बनाता है जिसे function stack कहते हैं ।
- यह function stack एक प्रकार के frames का stack होता है ।
- जिसमे हरबार कॉल करने पर नए parameter की वैल्यू को stack में पुश करता जाता है । इस stack के टॉप पर हमेशा last कॉल वाले parameter की वैल्यू रहती है ।
- जब base case true होता है तब stack से values को एक एक करके pop किया जाता है और जोड़ कर वापस उत्तर मिल जाता है ।

Recursive Function

- WAP to print a string backwards (using Recursion)

```
def backPrint(st,n):  
    if n>0:  
        print(st[n],end="")  
        backPrint(st,n-1)  
    elif n==0:  
        print(st[0])  
  
s=input("Enter a String")  
backPrint(s,len(s)-1)
```

```
= RESTART: C:/Users/SHAURYA/  
Enter a StringSanjeev  
veejnaS
```

Recursive Function

- WAP to calculate power e.g. a^n (using Recursion)

```
def power(a,b):  
    if b==0:  
        return 1  
    else:  
        return a*power(a,b-1)  
  
x=int(input("Enter value of x (Positive) : "))  
n=int(input("Enter value of n (Positive) : "))  
result=power(x,n)  
print("The result is : ",result)
```

```
= RESTART: C:/Users/SHAURYA/AppD  
Enter value of x (Positive) : 3  
Enter value of n (Positive) : 4  
The result is : 81
```

Recursive Function

- WAP to print Fibonacci series (using Recursion)

```
def fibo(n):  
    if n==1:  
        return 0  
    elif n==2:  
        return 1  
    else:  
        return fibo(n-1)+fibo(n-2)  
  
n=int(input("Enter the range"))  
for i in range(1,n+1):  
    print(fibo(i),end=", ")  
print("...")
```

```
= RESTART: C:/Users/SHAURYA  
Enter the range10  
0,1,1,2,3,5,8,13,21,34,...
```

Recursion v/s Iteration

- Recursion किसी solution को दिखने में छोटा बनाता है और गणितीय विधि से हल करता है ।
- Iteration एक लम्बी प्रक्रिया होती है जो loop पर पूर्णतया निर्भर करती है ।
- कौन सी विधि सही है recursion या iteration यह किसी भी प्रोग्राम की प्रकृति पर निर्भर करता है ।
- Recursive function, iteration की तुलना में slow होते हैं ।
- Iteration थोड़ा मेमोरी space अधिक लेता है ।

Assignment

1. Write a recursive code to compute and print sum of squares of n numbers. Value of n is passed as parameter.
2. Write a recursive code to compute greatest common divisor of two numbers.

धन्यवाद

और अधिक पाठ्य-सामग्री हेतु निम्न लिंक पर क्लिक करें -

www.pythontrends.wordpress.com

एक शुरुआत
pythontrends
पाइथन सीखें और सिखाएं

मुख्य पृष्ठ/Home
संपर्क/Contact
लेख/Articles
छायाचित्र/Images
कक्षा-11 कंप्यूटर साइंस/Class - XI Computer Science
कक्षा-11 आई० पी० /Class -XI IP
आदर्श प्रश्नपत्र/Sample Papers
उपयोगी लिंक्स / Useful Links

नमस्ते दोस्तों ! /Hello Friends!



यह ब्लॉग उन बच्चों की मदद के लिए बनाया गया है जो python में प्रोग्रामिंग सीख रहे हैं | यह ब्लॉग द्विभाषीय होगा जिससे सीबीएसई बोर्ड के वे बच्चे जिन्हें अंग्रेजी भाषा में समस्या होती है उन्हें सही मार्गदर्शन करेगा तथा प्रोग्रामिंग में उनकी सहायता करेगा | जैसा की हम जानते हैं की हमारे देश में कई क्षेत्र और कई लोग ऐसे हैं जिनकी अंग्रेजी उतनी मज़बूत नहीं है क्यों कि ये हमारी मातृभाषा नहीं है | तो हमें कभी कभी अंग्रेजी के कठिन शब्दों को

Customize ...

ENG 12:31 AM 10/30/2018